

Initialize unspecified aggregate members with direct list initialization

Document number: P0280R0
Date: 2016-02-08
Project: Programming Language C++, Evolution Working Group
Reply-to: James Touton <bekenn@gmail.com>

Table of Contents

1. [Table of Contents](#)
2. [Overview](#)
3. [Impact On the Standard](#)
4. [Technical Specifications](#)
5. [Wording](#)

Overview

The following program is ill-formed:

```
struct S
{
    explicit S(int a = 0) : _a{a} { }
    int _a;
};

int main()
{
    S arr[2] = { S{} };    // error : chosen constructor is explicit in copy-initialization
}
```

The error occurs when attempting to match a constructor for the second array element of `arr`. This is because aggregate initialization uses copy initialization for unspecified aggregate members (in this case, the second array element). Even though `S`'s constructor is a default constructor (thanks to the default argument), it is unavailable to copy initialization because it is marked `explicit`. This was most likely an unintended consequence of the wording for aggregate initialization. This paper proposes using direct initialization for unspecified aggregate members in this situation.

Impact On the Standard

This change fixes an obscure corner case, causing previously invalid code to become valid. It has no effect on currently valid code.

Technical Specifications

When an aggregate is initialized by an initializer list, and the initializer list contains fewer items than there are assignable members of the aggregate, the remaining unspecified members are initialized using direct list initialization from an empty list.

Wording

All modifications are presented relative to N4567.

Modify §8.5.1 [dcl.init.aggr] paragraph 7:

If there are fewer *initializer-clauses* in the list than there are members in the aggregate, then each member not explicitly initialized shall be initialized from its default member initializer (9.2) or, if there is no default member initializer, from an empty initializer list using direct-list-initialization (8.5.4).